

252-0027

Introduction to Programming Exercises

Week 1: IntelliJ and EBNF

Jan Leonard

**Department of Computer Science
ETH Zurich**

Plan for today

- About Me
- Organization
- Exercise Sheet 1 - IntelliJ Installation
- EBNF Recap and Exercises
- EBNF Extra Exercises
- Quick Survey
- Advice for EProg
- Mental Health

About me

- Name: Jan Leonard
- 7th Semester of Computer Science
- I want to create a dynamic exercise session
- **Please ask questions**

Typical Exercise Session

- Review of the last exercise
- Theory Recap
- A few exercises
- Preview of the next week
- Occasionally a Kahoot
- Your Questions

Organization

- For questions: *leonardj@ethz.ch*
Please put «[EProg26]» in the subject
- New exercises: **Tuesday evening** (normally)
- Submit exercises by **Tuesday evening (23:59)** following week
Always submit via Git
Solutions in a separate Git project

Organization

- Slides and additional materials on my website
- janleonard115.github.io/EPROG-HS-2026/



Eprog TA — HS 2026

janleonard115.github.io

Quick Note

- This group is for **non-native** German speakers
- There are repetition groups
 - Also reserved for students who are repeating EProg
- I am happy to have you here - no matter the reason
 - Non-native speakers get priority if we ever run out of space

Organization

- **Good News:** There will be an English version of the exam, the exercise slides and the exercises (including bonus)
- **BUT:** The German version is binding

IntelliJ already installed before?

- New version: **Java 25**

Install Java 25 from the Oracle archive -> select JRE 25 as the default in IntelliJ

Information on error messages and fixes directly on the website

Otherwise uninstall IntelliJ and Java, reinstall JDK 25 and IntelliJ

Very important, because grading the bonus exercises also uses Java 25



More on IntelliJ

IntelliJ Installation

- You can find the installation instructions in Exercise 1.
 - <https://lec.inf.ethz.ch/infk/eprog/2026/exercises/sheets/u01.pdf>

EBNF Recap

- **Form:** LHS <- RHS
 - **Example:** <digit_9> <- 9
- **LHS:** Name of the EBNF Rule
- **RHS:** Description of the name
 - Other names
 - Literals
 - Control Forms

EBNF Recap

- $\langle \rangle$ = Name of the EBNF Rule
- What is **epsilon** (ϵ)?
 - Empty Literal
 - We choose „nothing“

EBNF Notation

- In old exams, italics are often used for EBNF rules.
- *digit* instead of <digit>

EBNF Control Forms

- **Sequence:**
 - A B means A is followed by B
 - **Ignore the whitespaces**
 - Not as important anymore because the exam is on moodle
 - We will see Quizzes to prepare for the format

EBNF Control Forms

- **Decision:**
 - $A \mid B$ means either A or B can appear at that position - not both

EBNF Control Forms

- **Optional:**
 - **[A]** means A may appear 0 or 1 time

EBNF Control Forms

- Repetition:
 - $\{ A \}$ means A may appear 0 or more times

EBNF Control Forms

- **Applying** a rule
 - Replace the LHS by the RHS
- **Special rule:**
 - Starting rule of our derivation is the **last rule**

EBNF Parentheses

- **A | B C**
- Unclear if the language is:
 - A or BC
 - AC or BC
- We use parentheses to make it precise

How to solve EBNF Exercises - Preview

- **Derivation Table**

- First row is the start rule
- Last row is the resulting string
- Transition between two rows corresponds to one derivation step

- **Derivation Tree**

- The root is the name of the start rule
- The leaves are characters
- The edges represent a derivation step

Example: Derive -127 as a Tree

<digit>	←	0 1 2 3 4 5 6 7 8 9
<integer>	←	[+ -] <digit> {<digit>}

Example: Derive -127 as a Table

(R1)	<digit>	←	0 1 2 3 4 5 6 7 8 9
(R2)	<integer>	←	[+ -] <digit> {<digit>}

Kahoot!

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

1245

EBNF: Legal / Not Legal

Given the EBNF description of value

<i>digit</i>	$\Leftarrow$	<table><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table>	0	1	2	3	4	5	6	7	8	9		
0	1	2	3	4	5	6	7	8	9					
<i>separator</i>	$\Leftarrow$	<table><tr><td>-</td></tr></table>	-											
-														
<i>char</i>	$\Leftarrow$	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td></tr></table>	A	B	C	D	E	F	a	b	c	d	e	f
A	B	C	D	E	F	a	b	c	d	e	f			
<i>num</i>	$\Leftarrow$	<i>digit</i> { [<i>separator</i>] <i>digit</i> }												
<i>int</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> }												
<i>real</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } [<table><tr><td>.</td></tr></table> <i>digit</i> { <i>digit</i> }]	.											
.														
<i>cd</i>	$\Leftarrow$	<i>char</i> <i>digit</i>												
<i>hexa1</i>	$\Leftarrow$	<i>cd</i> { <i>cd</i> }												
<i>hexa2</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } <table><tr><td>h</td></tr></table>	h											
h														
<i>hexa</i>	$\Leftarrow$	<i>hexa1</i> <i>hexa2</i>												
<i>value</i>	$\Leftarrow$	<i>num</i> <i>real</i> <i>int</i> <i>hexa</i>												

00972

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

00100h

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

1a00

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

1a00h

EBNF: Legal / Not Legal

Given the EBNF description of value

<i>digit</i>	$\Leftarrow$	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table>	0	1	2	3	4	5	6	7	8	9		
0	1	2	3	4	5	6	7	8	9					
<i>separator</i>	$\Leftarrow$	<table border="1"><tr><td>-</td></tr></table>	-											
-														
<i>char</i>	$\Leftarrow$	<table border="1"><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td></tr></table>	A	B	C	D	E	F	a	b	c	d	e	f
A	B	C	D	E	F	a	b	c	d	e	f			
<i>num</i>	$\Leftarrow$	<i>digit</i> { [<i>separator</i>] <i>digit</i> }												
<i>int</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> }												
<i>real</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } [<table border="1"><tr><td>.</td></tr></table> <i>digit</i> { <i>digit</i> }]	.											
.														
<i>cd</i>	$\Leftarrow$	<i>char</i> <i>digit</i>												
<i>hexa1</i>	$\Leftarrow$	<i>cd</i> { <i>cd</i> }												
<i>hexa2</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } <table border="1"><tr><td>h</td></tr></table>	h											
h														
<i>hexa</i>	$\Leftarrow$	<i>hexa1</i> <i>hexa2</i>												
<i>value</i>	$\Leftarrow$	<i>num</i> <i>real</i> <i>int</i> <i>hexa</i>												

1_000_000

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

001ab.001h

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

209AB

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

4.9901

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

deadface

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

4_000.0

EBNF: Legal / Not Legal

Given the EBNF description of value

<i>digit</i>	$\Leftarrow$	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table>	0	1	2	3	4	5	6	7	8	9		
0	1	2	3	4	5	6	7	8	9					
<i>separator</i>	$\Leftarrow$	<table border="1"><tr><td>-</td></tr></table>	-											
-														
<i>char</i>	$\Leftarrow$	<table border="1"><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td></tr></table>	A	B	C	D	E	F	a	b	c	d	e	f
A	B	C	D	E	F	a	b	c	d	e	f			
<i>num</i>	$\Leftarrow$	<i>digit</i> { [<i>separator</i>] <i>digit</i> }												
<i>int</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> }												
<i>real</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } [<table border="1"><tr><td>.</td></tr></table> <i>digit</i> { <i>digit</i> }]	.											
.														
<i>cd</i>	$\Leftarrow$	<i>char</i> <i>digit</i>												
<i>hexa1</i>	$\Leftarrow$	<i>cd</i> { <i>cd</i> }												
<i>hexa2</i>	$\Leftarrow$	<i>digit</i> { <i>digit</i> } <table border="1"><tr><td>h</td></tr></table>	h											
h														
<i>hexa</i>	$\Leftarrow$	<i>hexa1</i> <i>hexa2</i>												
<i>value</i>	$\Leftarrow$	<i>num</i> <i>real</i> <i>int</i> <i>hexa</i>												

00100H

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

001ab.001

EBNF: Legal / Not Legal

Given the EBNF description of value

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

separator $\Leftarrow$ -

char $\Leftarrow$ A | B | C | D | E | F | a | b | c | d | e | f

num $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [. *digit* { *digit* }]

cd $\Leftarrow$ *char* | *digit*

hexa1 $\Leftarrow$ *cd* { *cd* }

hexa2 $\Leftarrow$ *digit* { *digit* } h

hexa $\Leftarrow$ *hexa1* | *hexa2*

value $\Leftarrow$ *num* | *real* | *int* | *hexa*

0x0ABC

Extra Exercises

- Create a description $\langle \text{palindrome} \rangle$, whose legal symbols allow all numbers that read the same from the front and the back and use only the digits from 1 to 4. Examples are 11, 232, 444
- Create a description $\langle \text{five} \rangle$, which permits all sums of positive numbers that add up to 5. Examples are “1 + 4”, “2 + 1 + 1 + 1”, “5”
- Create a description for $\langle \text{oddEight} \rangle$, which contains all numbers in which the digit 8 occurs an odd number of times.

Preview

- More EBNF
 - Recursion
 - Other Examples
- GitHub
- We will cover all of the above in the next exercise session



1. EProg Questions Group 15.1

docs.google.com

Advice for EProg

- **Do not underestimate EProg**
- Go to the exercise sessions
- Solve the exercises and try to get the bonus
- The other exercises are equally important

Advice for EProg

- **This is not a Java Course**
- Lots of Theory on Programming
- Logical Thinking and Problem Solving
- This is why LLMs will not help you much in this course

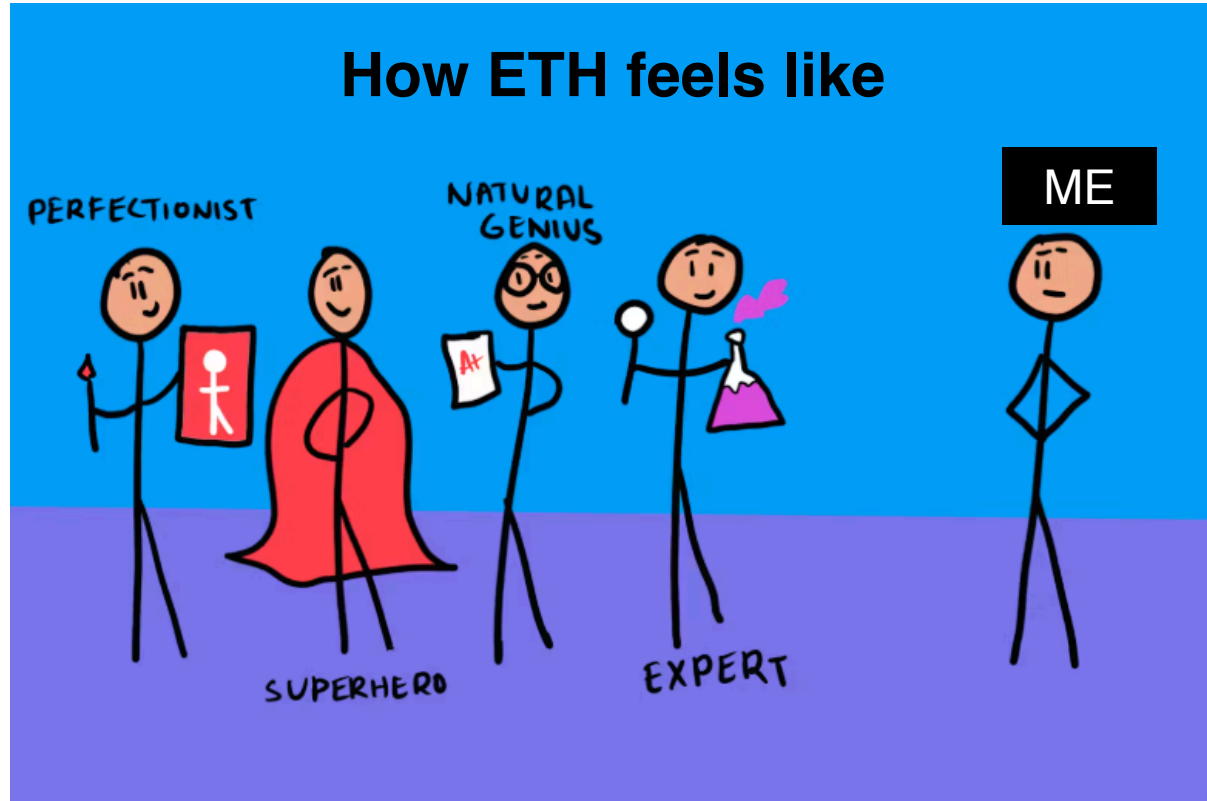
Mental Health during Basisjahr

- Long Stressful Semester
- There is always something to do

Mental Health during Basisjahr

- Try to keep the balance
- Prioritize sleep and nutrition
- Make time for friends
- Build in real breaks
- Ask for help

Imposter Syndrome



Imposter Syndrome

- Be honest with the people around you
- Surround yourself with supportive people
- Don't compare your inside to everyone else's outside
- You were accepted to ETH because you belong here

Thank you for your attention!

Have a great rest of the week!

